广州周立功单片机发展有限公司笔试题 A 卷参考答案

一、单选题

1	A	2	A	3	A	4	B	5	C
6	B	7	C	8	A	9	B	10	D
11	B	12	C	13	C	14	C	15	A
16	D	17	D	18	C	19	B	20	C
21	C	22	B	23	A	24	C	25	C

1、Linux 内核中宏 EXPORT_SYMBOL 和宏 EXPORT_SYMBOL_GPL 作用各是什么？

答：

1）EXPORT_SYMBOL标签内定义的函数或者符号对全部内核代码公开，不用修改内核代码就可以在您的内核模块中直接调用，即使用EXPORT_SYMBOL可以将一个函数以符号的方式导出给其他模块使用。

2）EXPORT_SYMBOL(name)和EXPORT_SYMBOL_GPL(name)这两个宏均用于将给定的符号导出到模块外。_GPL版本的宏定义只能使符号对GPL许可的模块可用。

2、Linux 内核中宏 container_of（ptr，type，member）的作用是什么？

答：

container_of(ptr,type,member) 用于在已知结构体里面成员member和该成员指针ptr（就是地址）和结构体类型type, 返回该成员所在的结构体的指针（就是地址）。

3、Linux 内核中 HZ 值是什么含义？

答：

时钟中断由系统定时硬件以周期性的间隔产生，这个间隔由内核根据 HZ 值来设定，HZ 是一个体系依赖的值，在 <linux/param.h>中定义或该文件包含的某个子平台相关文件中。

4、Linux 进程中的 PID ,PPID, PGID，SID 各是什么？

答：

1）Linux系统中总是会分配一个号码用于在其命名空间中唯一地标识它们，即进程ID号(PID)。

2）PPID是当前进程的父进程的PID。

3）PGID(Process Group ID 进程组 ID号)。

4）SID(Session ID 会话ID)。

5、kmalloc 与 vmalloc 有啥区别？

答：

1）kmalloc保证分配的内存在物理上是连续的,vmalloc保证的是在虚拟地址空间上的连续。

2）vmalloc比kmalloc要慢。

3）kmalloc能分配的大小有限，vmalloc能分配的大小相对较大。

4）一般情况下在驱动程序中都是调用kmalloc()来给数据结构分配内存，而vmalloc()用在为活动的交换区分配数据结构，为某些I/O驱动程序分配缓冲区，或为模块分配空间。

6、什么叫原子上下文，内核的哪些情形处于原子上下文？

广州周立功单片机发展有限公司笔试题 A 卷部分答案解析：

一、选择题

1、C

```c
#include <stdio.h>
#define N 2
#define M N+1
#define NUM (M+1)*M/2

int main(int argc, char *argv[])
{
    int i= 0;
    printf("NUM=%d\n", NUM);
    while(i < NUM)
        printf("%d", i++);
    printf("\n");

    return 0;
}
```

```
gec:Text$ sudo gcc demo.c -o demo
gec:Text$ ./demo
NUM=8
01234567
gec:Text$
```

2、A

```c
#include <stdio.h>

int mul(int a, int b, int c)
{
    return a*b*c;
}

int main(int argc, char *argv[])
{
    int (*function_pointer)(int, int, int);
    function_pointer = mul;
    printf("The product of three numbers is: %d",
                    function_pointer(2, 3, 4));

    return 0;
}
```

```
gec:Text$ sudo gcc demo.c -o demo
gec:Text$ ./demo
The product of three numbers is: 24gec:Text$
```

3、A

```c
#include <stdio.h>
int x = 5;

void n()
{
    printf("%d", x);
}

void m()
{
    x = 8;
    n();
}

int main(int argc, char *argv[])
{
    int x = 3;
    m();
    printf("%d", x);
    return 0;
}
```

```
gec:Text$ sudo gcc demo.c -o demo
gec:Text$ ./demo
83gec:Text$
```

4、B

```c
#include <stdio.h>
int main(int argc, char *argv[])
{
    float x = 0.1;
    if(x == 0.1)
        printf("Equal");
    else
        printf("Not equal");
    return 0;
}
```

```
gec:Text$ sudo gcc demo.c -o demo
gec:Text$ ./demo
Not equalgec:Text$
```

5、C

```c
#include <stdio.h>

struct A{
    int i;
    char j;
    char *ptr;
    long Arry[100];
    char b[2];
    char *c;

}A1;

int main(int argc, char *argv[])
{
    printf("%d\n", sizeof(A1));
    return 0;
}
```

```
gec:Text$ sudo gcc demo.c -o demo
gec:Text$ ./demo
420
gec:Text$
```

6、B

```c
#include <stdio.h>

int main(int argc, char *argv[])
{
    int a[10]={1, 2, 3, 4, 5, 6, 7, 8, 9, 10}, *p=a;
    printf("*p= %d\n", *p);//*p=a[0]=1
    printf("*a= %d\n", *a); //*a=a[0]=1
    printf("a[10]= %d\n", a[10]);//数组的越界访问
    printf("3[p]= %d\n", 3[p]);//3[p]=*(p+3)=4
    return 0;
}
```

```
gec:Text$ sudo gcc demo.c -o demo
gec:Text$ ./demo
*p= 1
*a= 1
a[10]= -1075354940
3[p]= 4
gec:Text$
```

7、C

8、A

9、B

10、D

```c
#include <stdio.h>

int sum(int n)
{
    if(n <= 1)
        return n;
    else
        return sum(n-1)+n;
}

int main(int argc, char *argv[])
{
    int ret= sum(4);
    printf("sum= %d\n", ret);
    return 0;
}
```

```
gec:Text$ sudo gcc demo.c -o demo
gec:Text$ ./demo
sum= 10
gec:Text$
```

11、B

```c
#include <stdio.h>

int func(int x)
{
    int countx = 0;
    while(x)
    {
        countx++;
        x = x&(x-1);
    }
    return countx;
}

int main(int argc, char *argv[])
{
    int ret= func(0x22B8);
    printf("ret= %d\n", ret);
    return 0;
}
```

```
0x22B8H=0010 0010 1011 1000B(x)
  &     0010 0010 1011 0111B(x-1)
---------------------------------
        0010 0010 1011 0000B
    &   0010 0010 1010 0000B
---------------------------------
        0010 0010 1010 0000B
    &   0010 0010 1001 0000B
---------------------------------
        0010 0010 1000 0000B
    &   0010 0010 0111 0000B
---------------------------------
        0010 0010 0000 0000B
    &   0010 0001 0000 0000B
---------------------------------
        0010 0000 0000 0000B
    &   0001 0000 0000 0000B
---------------------------------
        0000 0000 0000 0000B→0D→X
    →countx = 6
```

```
gec:Text$ sudo gcc demo.c -o demo
gec:Text$ ./demo
ret= 6
gec:Text$
```

12、C

```cpp
#include <iostream>
using namespace std;

class B{
    public:
        virtual int shift(int n=2) const {return n<<2;}
};

class D:public B{
    public:
        int shift(int n) const {return n<<3;}
};

int main(int argc, char *argv[])
{
    const D d;
    const B *b = &d;

    std::cout << b->shift() << std::endl;
    return 0;
}
```

```
gec:Text$ sudo g++ demo.cpp -o demo
[sudo] password for gec:
gec:Text$ ./demo
16
gec:Text$
```

13、C

```cpp
#include <iostream>
using namespace std;

class CTestClass{
    public:
        CTestClass(){}
        ~CTestClass(){}
    static int m_nTest;
};

int CTestClass :: m_nTest = 100;

int main(int argc, char *argv[])
{
    CTestClass tc1, tc2, tc3;
    tc1.m_nTest = 101;
    tc2.m_nTest = 102;
    cout << tc3.m_nTest << endl;

    return 0;
}
```

```
gec:Text$ sudo g++ demo.cpp -o demo
gec:Text$ ./demo
102
gec:Text$ ./demo
102
```

14、C
15、A
16、D
A） 线程只能在进程内部重新分配 CPU 资源,或者影射为系统线程才能参与全局 CPU 资源的竞争
B）线程不拥有系统资源，但是占用 CPU 资源。
17、D
[解析] MAC 地址也叫物理地址、硬件地址或链路地址，MAC 地址由网络设备制造商生产时写在硬件内部。IP 地址与 MAC 地址在计算机里都是以二进制表示的，IP 地址是 32 位的，MAC 地址是 48 位的。
18、C
A）传输层；B）网络层；D）传输层
19、B
20、C
21、C
22、B
23、A
24、C
25、C

二、岗位方向题目

1、答：
1）EXPORT_SYMBOL 标签内定义的函数或者符号对全部内核代码公开，不用修改内核代码就可以在您的内核模块中直接调用，即使用 EXPORT_SYMBOL 可以将一个函数以符号的方式导出给其他模块使用。
2）EXPORT_SYMBOL(name)和 EXPORT_SYMBOL_GPL(name)这两个宏均用于将给定的符号导出到模块外. _GPL 版本的宏定义只能使符号对 GPL 许可的模块可用。

2、答：
container_of(ptr,type,member) 用于在已知结构体里面成员 member 和该成员指针 ptr（就是地址）和结构体类型 type, 返回该成员所在的结构体的指针（就是地址）。

3、答：
时钟中断由系统定时硬件以周期性的间隔产生，这个间隔由内核根据 HZ 值来设定，HZ 是一个体系依赖的值，在 <linux/param.h>中定义或该文件包含的某个子平台相关文件中。

4、答：
1)Linux 系统中总是会分配一个号码用于在其命名空间中唯一地标识它们,即进程 ID 号(PID)。
2）PPID 是当前进程的父进程的 PID。
3）PGID(Process Group ID 进程组 ID 号)。
4）SID(Session ID 会话 ID)。

5、答：

1）kmalloc 保证分配的内存在物理上是连续的,vmalloc 保证的是在虚拟地址空间上的连续。

2）vmalloc 比 kmalloc 要慢。

3）kmalloc 能分配的大小有限， vmalloc 能分配的大小相对较大。

4）一般情况下在驱动程序中都是调用 kmalloc()来给数据结构分配内存，而 vmalloc()用在为活动的交换区分配数据结构，为某些 I/O 驱动程序分配缓冲区，或为模块分配空间。

6、答：

一般来讲原子上下文指的是在中断或软中断中，以及在持有自旋锁的时候。内核不能访问用户空间，而且内核是不能睡眠的。在这种情况下，就是内核的原子上下文了。

情况：

1）#define in_irq()　　(hardirq_count()) //在处理硬中断中

2）#define in_softirq()　　(softirq_count()) //在处理软中断中

3）#define in_interrupt()　　(irq_count()) //在处理硬中断或软中断中

4）#define in_atomic()　　((preempt_count() & ~PREEMPT_ACTIVE) != 0) //包含以上所有情况